

Département GM

RAPPORT PROJET MIPP

Jeu des chiffres croisés

Encadrant:
M. A. OSMANI

Présenté par:
Mouad SHERADJ
DRISSI
Amr ZAKI SALIH

Année universitaire: 2024-2025

Table des matières

1	Introduction	2
2	Cahier des charges	2
2.1	Fonctionnalités et exigences	2
3	UML Initial	4
3.1	Diagramme de Classe	4
3.2	Diagramme de cas d'utilisation	7
3.3	Diagramme de séquence	8
3.4	Diagramme d'objet	9
4	Implémentation Java	10
4.1	Outils utilisés	10
4.2	Architecture générale du projet	11
4.3	Classes principales	11
5	UML après code	19
6	Test	20
7	Conclusion	27

1 Introduction

Le but de notre projet informatique intitulé « Jeu des chiffres croisés » était de développer un jeu interactif mêlant logique et calcul mental, inspiré des mots croisés mais basé sur des opérations arithmétiques. Pour ce faire, nous avons commencé par modéliser l'idée principale à l'aide d'un diagramme UML, afin de mieux visualiser les différentes classes nécessaires ainsi que leurs interactions. Une fois cette étape terminée, nous avons procédé à l'implémentation en langage Java.

Ainsi, dans ce rapport nous commencerons par présenter le cahier des charges, puis plusieurs diagrammes UML, puis reviendrons sur le code puis une amélioration des diagrammes d'UML initiaux et enfin nous reviendrons sur les tests finaux liés au code du jeu.

2 Cahier des charges

Le but du projet est de réaliser un programme qui simule le jeu des chiffres croisés dans le langage de programmation orienté objet Java. Les règles du jeu sont : trois cartes numérotées sont tirées au hasard et leurs valeurs deviennent les trois nombres cibles pour toute la partie, c'est-à-dire les nombres que les joueurs doivent atteindre lorsqu'ils placent les jetons sur le plateau. Les jetons posés doivent s'intégrer aux jetons déjà posés et on peut en échanger 1, 2 ou 3 avec ceux de la pioche en cas d'impossibilité de pose.

2.1 Fonctionnalités et exigences

2.1.1 Gestion de la partie

- Démarrage d'une partie
 - Tirage aléatoire des 3 nombres cibles pour toute la partie
 - Initialisation du plateau de jeu vide
 - Attribution des 6 jetons numérotés pour chaque joueur
- Fin de partie
 - La partie se termine quand on ne peut plus poser de jetons ou la sélection d'un joueur de l'option fin du jeu
 - Détermination du gagnant (le joueur ayant obtenu le plus de point)

2.1.2 Mécaniques du jeu

- Placement des jetons
 - Le joueur peut poser ses jetons sur le plateau s'ils s'intègrent aux jetons déjà posés
 - Vérification que le placement respecte les règles du jeu

- Échange de jetons
 - Si un joueur ne peut pas jouer, il peut échanger 1,2 ou 3 jetons avec ceux du sac à jetons
 - Mise à jour de son chevalet après échange
- Validation des combinaisons
 - Vérification que la somme des jetons dans une ligne/colonne atteint un des nombres cibles
 - Calcul de points(nombres de points = valeur de la carte + nombre de jetons utilisés)

2.1.3 Interface et interaction

- Affichage du plateau
 - Représentation du plateau de jeu
 - Affichage des chevalets avec jetons disponibles pour chaque joueur(max 9 jetons)
- Tour par tour
 - Désignation du tour d'un joueur
 - Proposition d'actions : poser un jeton, échanger,fin de partie.

2.1.4 Contraintes

- Le jeu doit être développé en Java
- Plateau de 17x17 cases représenté par un tableau 2D
- Sac jeton composé de 90 jetons, numérotés de 1 à 20
- 36 possibilités pour les cartes numérotés de 15 à 50(nombres cibles)

2.1.5 Validation

Conditions :

- Le programme doit prendre aléatoirement les 3 nombres cibles et les 6 jetons de chaque joueur dans la liste de cartes et le sac de jetons respectivement
- Les joueurs doivent pouvoir placer leurs jetons sur un plateau en respectant les règles du jeu
- Un système de vérification doit confirmer si une combinaison atteint un nombre cible
- L'interface doit afficher clairement le plateau, les jetons et le tour en cours ainsi que la liste des scores
- Implémentation d'un système d'échange en cas d'impossibilité de pose
- Une partie doit se terminer correctement, avec détermination du gagnant

3 UML Initial

Pour pouvoir mener à bien le projet, il était nécessaire dans un premier temps de réaliser un diagramme de classe pour bien faciliter la conception du code. Nous avons ainsi réalisé 4 diagrammes initiaux, un diagramme de classe, un diagramme de séquence, un diagramme d'objet ainsi qu'un diagramme de cas d'utilisation. Le premier cité permet de modéliser les classes et leurs relations, le deuxième quant à lui permet de montrer le déroulement dans le temps des interactions entre les objets et permet de compléter le diagramme de classe, le diagramme d'objet permet de visualiser une situation concrète du programme en montrant des objets réels, leurs valeurs et leurs relations, et enfin le dernier diagramme permet de représenter les fonctionnalités principales du système du point de vue de l'utilisateur ou des acteurs extérieurs.

3.1 Diagramme de Classe



Figure 1.1 – *Diagramme de Classe initial*

Les 7 classes initiales suivantes permettent la modélisation du **jeu des mots croisés**.

Classe Joueur

- **Attributs :**
 - Nom : String
 - Score : int
 - chevalet : Chevalet
- **Méthodes :**
 - GetNom()
 - SetNom(String Nom)
 - GetScore()
 - SetScore(int Score)
 - Jouer()
 - Piocher()

La classe **Joueur** permet la représentation d'un joueur en jeu. Un joueur possède un nom, un score, un chevalet et peut jouer et piocher. **Piocher** permet de piocher au départ un nombre de jetons à mettre dans le chevalet du joueur et **Jouer** permet de poser sur le plateau les jetons souhaités.

3.1.1 Classe Chevalet

- **Attributs :**
 - ListeDeJetons
- **Méthodes :**
 - EchangerJetons(liste de jeton)
 - GetNombreDeJetons()
 - UtiliserJetons()
 - Piocher()

La classe **Chevalet** représente un chevalet qui contient plusieurs jetons. Les méthodes **EchangerJetons**, **UtiliserJetons** ainsi que **Piocher** changent l'attribut **ListeDeJetons**. **EchangerJetons** permet d'échanger un nombre de jetons avec ceux piochés dans le sac à objet. **UtiliserJetons** quant à lui permet d'utiliser les jetons et **Piocher**, pioche le nombre de jetons à mettre au départ dans le chevalet.

Classe Systeme

- **Attributs :**
 - ListeJoueur
- **Méthodes :**
 - GestionDebutFin(Paquet p)
 - CalculerPoints()
 - GestionDesToursJoueurs()
 - Affichage()
 - MiseAJourPoint()
 - GetListeJoueur()
 - SetListeJoueur()

La classe **Systeme** permet tout ce qui est de l'ordre de la gestion globale du jeu. Elle permet par la méthode **GestionDebutFin** de gérer le début et la fin de la partie. Avec pour le début chacun des joueurs piochant et saisissant un nom de joueur ainsi que la pioche initiale des trois cartes et pour la fin de partie, un arrêt lorsque le plateau est plein ou alors si l'un des joueurs souhaite arrêter la partie. Il y a aussi la méthode **CalculerPoints** ainsi que **MiseAJourPoint** qui réalisent le calcul des scores ainsi que la mise à jour de ces derniers. Il est important de noter aussi **Affichage** qui permet l'affichage des scores, des chevalets de chacun des joueurs ainsi que du plateau.

Classe Carte

- **Attributs :**
 - numero : int
- **Méthodes :**
 - GetNumero()

La classe **Carte** permet de créer les cartes qui seront tirées lors du début de la partie.

Classe Jeton

- **Attributs :**
 - valeur
- **Méthodes :**
 - GetValeur()

La classe **Jeton** permet de représenter les jetons qui seront piochés lors de la pioche, qui seront stockés dans le chevalet ainsi que dans le sac à jetons.

Classe SacJetons

- **Attributs :**
 - ListeJeton

La classe **SacJetons** permet de représenter quant à elle le sac jetons qui est composé de 90 jetons, numérotés de 1 à 20

Classe Plateau

- **Méthodes :**
 - AjouterJeton(liste de jeton)
 - Verification()
 - MiseAJourPlateau()

La classe **Plateau** permet de représenter le plateau de jeton dans lequel il sera possible d'ajouter des jetons avec la méthode **AjouterJetons** et de vérifier si le coup est valide avec **Verification**. Le plateau est mis à jour grâce à la méthode **MiseAJourPlateau**;

3.2 Diagramme de cas d'utilisation



Figure 1.2 – *Diagramme de cas d'utilisation*

Ce diagramme représente les principales interactions entre le joueur et le système. Le joueur représenté à gauche et l'acteur qui initie la partie en lançant le jeu. Une fois la partie commencée, trois cartes sont tirées pour déterminer les cartes cibles. Le joueur

pioche ensuite six jetons, qu'il pourra poser sur le plateau en respectant les règles du jeu. Chaque fois que le joueur posera ses jetons, une vérification de l'emplacement ainsi qu'un calcul du score aura lieu. Le joueur peut aussi choisir à la place d'échanger des jeton au cours de la partie. Le jeu se termine lorsque les conditions de fin sont remplies. Ce diagramme de cas d'utilisation permet ainsi de visualiser les différentes étapes d'une partie ainsi que les fonctionnalités que le système doit gérer.

3.3 Diagramme de séquence

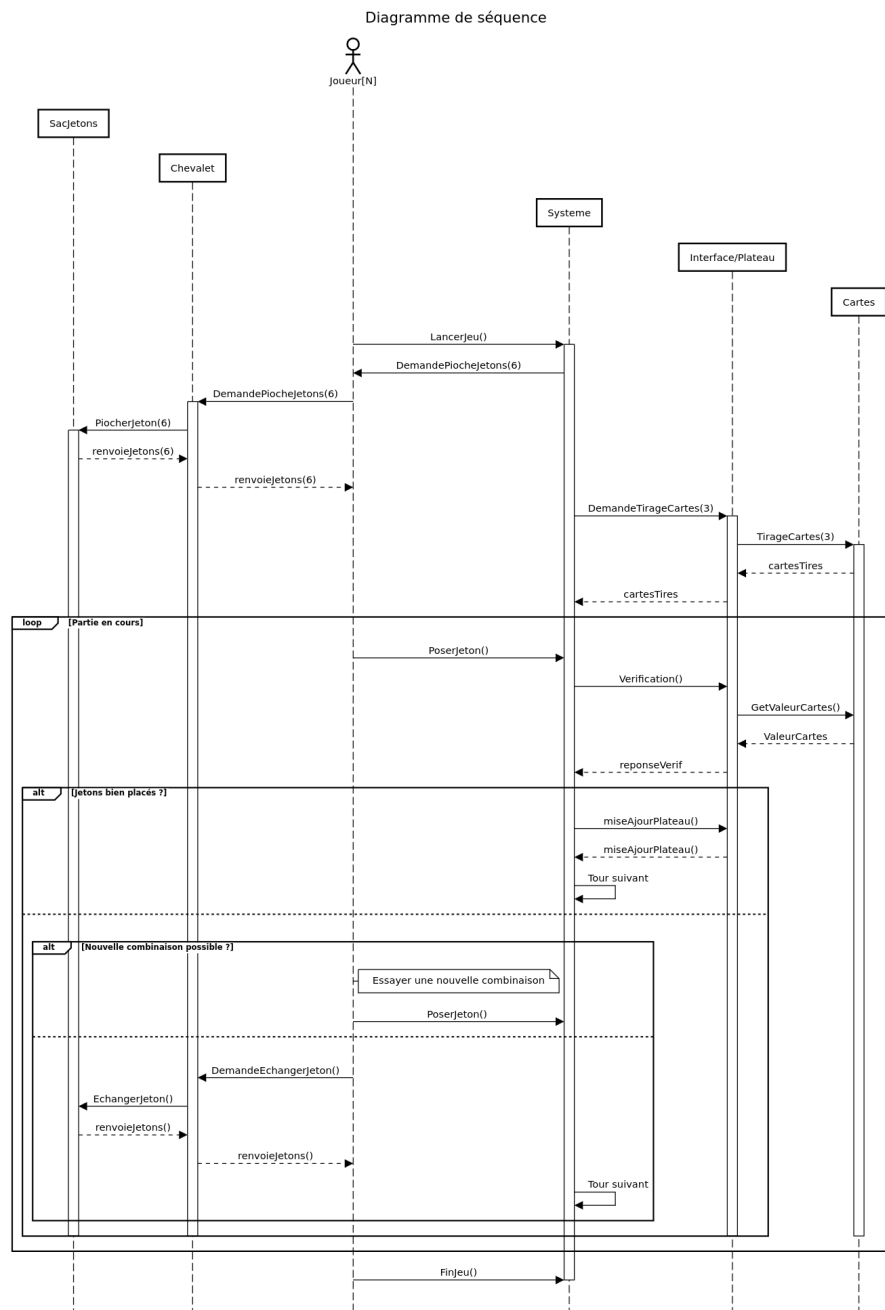


Figure 1.3 – *Diagramme de séquence*

Ce diagramme illustre le déroulement d'une partie en termes d'interactions entre les différents objets du système et le joueur. Il commence par l'action du joueur qui lance la partie. Le système envoie alors une requête de pioche de six jetons, récupérés depuis le sac et qui sont alors placés sur le chevalet du joueur. En parallèle une demande de tirage de trois cartes est envoyée puis les cartes sont tirées. Ensuite, la partie se déroule. A chaque tour, le joueur pose un jeton ce qui déclenche une vérification de son emplacement. Si les jetons sont bien posés le système passe au tour suivant. Si la combinaison n'est pas correcte, une alternative est proposée où le joueur peut essayer une nouvelle combinaison ou échanger ses jetons. Ce processus se répète jusqu'à que les conditions de fin de partie soient réunies.

3.4 Diagramme d'objet

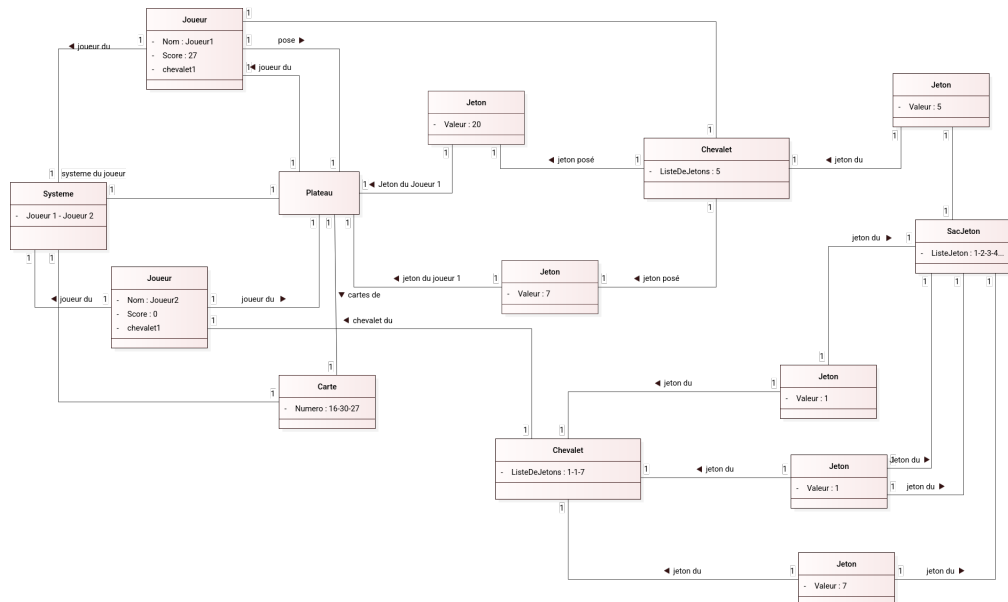


Figure 1.4 – *Diagramme d'objet*

Ce diagramme représente le programme pendant une exécution concrète du système. Ici est présenté deux joueurs, le joueur 1 et le joueur 2 avec chacun possédant plusieurs jetons

dans leurs chevalets respectifs. Les cartes demandées possèdent le numéro : 16-27-30. Les jetons posés par le joueur 1 sont le 7 et le 20. Il a dès lors un score de 29. Le diagramme d'objet permet contrairement au diagramme de classe qui est très générique d'exposer une instance réelle d'un scénario du jeu des chiffres croisés.

4 Implémentation Java

4.1 Outils utilisés

Dans le cadre de notre projet, nous avons utilisé un ensemble de packages qui ont facilité le processus de développement. Dans cette section, nous allons présenter ces packages ainsi que les principales classes et méthodes utilisées dans notre code.

4.1.1 Scanner

La classe `Scanner` provient du package `java.util`. Elle permet de lire des données saisies par l'utilisateur depuis le terminal, qu'il s'agisse de nombres entiers (`int`), de nombres à virgule flottante (`float`, `double`) ou de chaînes de caractères (`String`).

Les principales méthodes de `Scanner` que nous avons utilisées sont :

- `Scanner scanner = new Scanner(System.in)` : création d'un objet `Scanner` associé à l'entrée standard (le clavier).
- `scanner.nextLine()` : lecture d'une ligne complète de texte.
- `scanner.next()` : lecture du prochain mot saisi.
- `scanner.hasNextInt()` : vérifie si la prochaine entrée est un entier.
- `scanner.nextInt()` : lecture d'un entier saisi.

4.1.2 List et ArrayList

Les interfaces `List` et la classe `ArrayList` proviennent du package `java.util`. Elles permettent de stocker et manipuler des collections dynamiques d'objets.

Les principales méthodes de `List` et `ArrayList` que nous avons utilisées sont :

- `List<Jeton> listeJetons = new ArrayList<>()` : création d'une liste dynamique d'objet de classe jetons.
- `listeJetons.add(j)` : ajout d'un objet à la fin de la liste.
- `listeJetons.addAll(listeJetonsSelect)` : ajout de tous les éléments d'une autre liste.
- `listeJetons.remove(indice)` : suppression de l'élément à l'indice spécifié.
- `listeJetons.get(indice)` : récupération de l'élément à l'indice indiqué.
- `listeJetons.isEmpty()` : vérifie si la liste est vide.
- `for (Jeton j : listeJetons)` : parcours de la liste avec une boucle `for`.

4.1.3 Swing et Awt

Les packages `javax.swing` et `java.awt` permettent de créer des interfaces graphiques en Java. Ils sont principalement utilisés dans la classe `Plateau` pour construire le plateau de

jeu.[6]

Les composants graphiques principaux que nous avons utilisés sont :

- **JFrame** : crée une fenêtre principale.
- **JPanel** : conteneur pouvant contenir d'autres composants.
- **JLabel** : permet d'afficher du texte (ici, utilisé pour les jetons ou les cartes).
- **BoxLayout**, **GridLayout**, **BorderLayout** : différents gestionnaires de mise en page pour organiser les composants.
- **Font**, **Color**, **Dimension** : classes pour personnaliser le style et la disposition.
- **BorderFactory** : permet de créer différentes bordures autour des composants graphiques.

Ces outils nous ont permis d'implémenter une interface utilisateur interactive et intuitive pour le fonctionnement du jeu.[4]

4.2 Architecture générale du projet

Nous avons implémenté la modélisation présentée précédemment à l'aide des classes décrites dans la section 3.1.1. Dans un premier temps, nous avons développé les classes élémentaires *Jeton* et *Carte*, puis les classes *SacJetons* et *PaquetCartes*, représentant respectivement un ensemble de jetons et un ensemble de cartes. Ensuite, nous avons conçu la classe principale *Chevalet*, qui correspond à l'ensemble des jetons possédés par un joueur. Cette classe permet de piocher des jetons depuis le *SacJetons*, à l'initialisation du jeu ou d'échange de jetons avec ce dernier.

Par la suite, nous avons codé les classes principales qui gèrent la logique du jeu, à savoir : *Joueur*, qui représente un joueur avec ses actions et son score, et *Plateau*, qui gère l'affichage et enfin *Systeme* pour l'organisation du jeu.

Le projet est structuré en trois packages :

- **projet.src** : contient les classes liées à la gestion du jeu (*Joueur*, *Chevalet*, etc.).
- **projet.materiel** : regroupe les classes représentant les éléments matériels du jeu (*SacJetons*, *PaquetCartes*, *Jeton*, *Carte*, *Plateau*).
- **projet.utilisateur** : regroupe les classes spécifiques aux interactions avec le joueur.

Enfin, un fichier **main.java**, situé à la racine du projet, contient les tests unitaires et permet de lancer le jeu.

4.3 Classes principales

Dans cette partie, nous allons décrire les classes principale de notre jeu en expliquant leur attribues et le fonctionnement de leur méthodes.

4.3.1 Chevalet

La classe **Chevalet** représente l'ensemble des jetons détenus par un joueur. Elle est chargée de la gestion des jetons, notamment la pioche, l'échange et l'affichage. Chaque

chevalet contient au maximum neuf jetons. Elle interagit avec la classe `SacJetons` afin de permettre ces opérations.

— *Attributs principaux :*

- `listeJetons` : liste des jetons actuellement détenus par le joueur.
- `listeJetonspiochespourjouer` : jetons sélectionnés pour être utilisés lors du tour en cours.
- `sac_jetons` : référence au sac commun contenant les jetons restants.
- `capacite` : capacité maximale du chevalet (fixée à 9).

La méthode `echangerJetons()` permet à un joueur d'échanger une partie de ses jetons contre deux nouveaux, tirés aléatoirement dans le sac. Le processus commence par le tirage de deux jetons choisis aléatoirement dans le sac, qui sont ensuite temporairement retirés de celui-ci. Ces deux nouveaux jetons sont affichés à l'écran pour que le joueur puisse les voir.

Ensuite, le joueur est invité à saisir le nombre de jetons qu'il souhaite échanger, avec une limite maximale de trois. Plusieurs vérifications sont effectuées afin d'assurer que le nombre saisi est bien un entier valide, que la capacité maximale du chevalet n'est pas dépassée, et que le joueur respecte les règles du jeu.

Une fois le nombre de jetons validé, le joueur sélectionne les jetons qu'il souhaite échanger en indiquant leurs indices. Des contrôles supplémentaires permettent d'éviter les doublons ou les indices hors limites. Les jetons choisis sont alors retirés du chevalet et ajoutés au sac. En contrepartie, les deux jetons piochés au début sont ajoutés au chevalet du joueur. Ainsi, le contenu du chevalet est mis à jour correctement.

Cette méthode permet d'introduire une dimension stratégique dans le jeu.

Code Java de la méthode `echangerJetons()` :

```
1 public void echangerJetons(){
2     List<Jeton> listeJetonspioches = new ArrayList<>();
3     List<Jeton> listeJetonsSelect = new ArrayList<>();
4     Scanner scanner = new Scanner(System.in); //important classe Scanner du package
5         util
6     int verif;
7     for (int i=0;i<2;i++){
8         int indice_aleatoire = (int) (Math.random() * sac_jetons.getTaille());
9         listeJetonspioches.add(sac_jetons.getJeton(indice_aleatoire));
10        sac_jetons.supprimerJeton(indice_aleatoire);
11    }
12    System.out.println("La pioche : ");
13    listeJetonspioches.get(0).afficheVal();
14    listeJetonspioches.get(1).afficheVal();
15    System.out.println("Selectionner vos jetons");
16
17
18    do{
19
20        System.out.println("Saisir le nombre de jetons que vous voulez echanger : ");
```

```

21     afficherChev();
22     while (!scanner.hasNextInt()) {
23         System.out.println("Erreur : veuillez entrer un entier valide !");
24         scanner.next(); // On ignore l'entre incorrecte
25         System.out.println("Entrez un entier : ");
26     }
27
28     int nbr_jetons = scanner.nextInt();
29     verif= 2-nbr_jetons+getNombreJetons();
30     if(verif>this.capacite || nbr_jetons>3 || nbr_jetons<=0){
31         System.out.println("Erreur : Depassement capacite(9)");
32         verif=10;
33     }else{
34         for(int i=0;i<nbr_jetons;i++){
35
36             System.out.println("Saisir l'indice du jeton : ");
37             afficherChev();
38             while (!scanner.hasNextInt()) {
39                 System.out.println("Erreur : veuillez entrer un entier valide !");
40                 scanner.next();
41                 System.out.println("Entrez un entier : ");
42             }
43             int indice_jetons = scanner.nextInt()-1;
44             if (indice_jetons<0 || indice_jetons>nbr_jetons){
45                 System.out.print("Erreur choisissez dans l'intervalle");
46                 i-=1;
47             }else{
48
49                 listeJetonsSelect.add(this.listeJetons.get(indice_jetons));
50                 this.listeJetons.remove(this.listeJetons.get(indice_jetons));
51             }
52             listeJetons.addAll(listeJetonspioches);
53             sac_jetons.ajoutJetons(listeJetonsSelect);
54             break;
55         }
56     }
57     }while(verif>this.capacite);
58
59 }

```

4.3.2 Joueur

La classe Joueur représente un joueur dans la partie. Elle contient les informations essentielles telles que le nom du joueur, son score, son chevalet ainsi que la liste des positions de jetons posés durant un tour. Elle interagit avec les classes Chevalet, Plateau et SacJetons pour effectuer les différentes actions de jeu.

- *Attributs principaux :*
 - nom : nom du joueur.
 - score : score atteint par le joueur.
 - chevalet : chevalet de classe Chevalet associer au joueur.
 - plateau : référence du plateau de jeu.

- `sac_jetons` : référence au sac commun contenant les jetons restants.
- `nb_jetons_poses` :compteur des jetons utilises par le joueur.

La méthode `jouer()` permet à un joueur de poser des jetons sur le plateau à partir de son chevalet. Ce processus commence après que le joueur ait sélectionné les jetons à utiliser via la méthode `utiliser_jetons()` du chevalet. Ensuite, le joueur est invité à choisir, parmi les jetons sélectionnés, ceux qu'il souhaite placer sur le plateau.

À chaque tour, tant que le joueur n'a pas saisi la valeur zéro, et qu'il reste des jetons disponibles pour le tour tout en affichant les jetons sélectionnés. Le joueur choisit un jeton en saisissant son indice, puis saisit la position de destination sur le plateau à l'aide des coordonnées ligne et colonne.

La méthode vérifie que l'indice est valide. Si l'indice est correct, la grille du plateau de jeu est modifier, le jeton est retiré de la liste des jetons à jouer et le compteur interne `nb_jetons_poses` est incrémenté.

Une fois la boucle terminée (soit par l'utilisation de tous les jetons, soit par la sortie du joueur), les jetons non utilisés sont replacés sur le chevalet et la liste des jetons à jouer est vidée.

Code Java de la méthode `jouer()` :

```

1 public void jouer() {
2     Scanner scanner = new Scanner(System.in);
3     int indice = 0;
4     nb_jetons_poses = 0;
5
6     while (indice != -1 && !(chevalet.listeJetonspiochespourjouer.isEmpty())) {
7         chevalet.afficherjetonspioches();
8         System.out.print("Quelle pice voulez vous choisir ? (de 1 a "+ chevalet.
9             listeJetonspiochespourjouer.size() + " 0 pour sortir)");
10
11         while (!scanner.hasNextInt()) {
12             System.out.println("Erreur : veuillez entrer un entier valide !");
13             scanner.next();
14         }
15         indice = scanner.nextInt() - 1;
16         if (indice == -1) break;
17         else if (indice < 0 || indice >= chevalet.listeJetonspiochespourjouer.size()) {
18             System.out.println("Erreur, choisissez dans l'intervalle !");
19             continue;
20         }
21
22         int jeton_choisi = chevalet.listeJetonspiochespourjouer.get(indice).getValeur()
23             ;
24         System.out.print("Ecrivez la ligne et colonne");
25         while (!scanner.hasNextInt()) {
26             System.out.println("Erreur : veuillez entrer un entier valide !");
27             scanner.next();
28         }
29         int scanneri = scanner.nextInt();
30         while (!scanner.hasNextInt()) {

```

```

29         System.out.println("Erreur : veuillez entrer un entier valide !");
30         scanner.next();
31     }
32     int scannerj = scanner.nextInt();
33
34     plateau.modifierCase(scannerj, scannerj, jeton_choisi);
35     System.out.println(plateau.coupOK);
36     if (plateau.coupOK == true) {
37         chevalet.listeJetonspiochespourjouer.remove(indice);
38         nb_jetons_poses += 1;
39     }
40     plateau.affichage_bof();
41 }
42
43 if (!(chevalet.listeJetonspiochespourjouer.isEmpty())) {
44     chevalet.listeJetons.addAll(chevalet.listeJetonspiochespourjouer);
45     chevalet.listeJetonspiochespourjouer.clear();
46 }
47 }

```

4.3.3 Système de Jeu

Le système de jeu est responsable de la gestion du déroulement du jeu, de la gestion des joueurs et du calcul des points. Il se compose principalement de deux fonctionnalités majeures : la gestion des tours des joueurs et la vérification des tour en fonction des actions effectuées sur le plateau.

La méthode `gestionDesToursJoueurs()` permet de gérer les tours des différents joueurs. Tant que la partie est en cours et que des coups sont possibles, chaque joueur joue à tour de rôle. À chaque tour, le joueur choisit soit d'échanger des jetons, soit de jouer. Si le coup est valide, son score est mis à jour. Si le coup est invalide, les jetons joués sont retirés du plateau et rendus au joueur. Si un joueur n'a plus de jetons, il est éliminé. Lorsque le sac de jetons est vide, les joueurs doivent indiquer s'ils peuvent encore jouer ; sinon la partie se termine. À la fin de la boucle principale, une méthode vérifie si la partie peut continuer, puis une autre gère la fin de partie.

Code Java de la méthode `gestionDesToursJoueurs()` :

```

1 public void gestionDesToursJoueurs() {
2     boolean partieEnCours = true;
3     boolean couppossible = true;
4     Scanner scanner = new Scanner(System.in);
5
6     while (couppossible && partieEnCours) {
7
8         for (int i = 0; i < this.listeJoueur.size(); i++) {
9             plateau.affichage(this.listeJoueur, this.listeJoueur.get(i));
10            System.out.println("\nAu tour du joueur " + this.listeJoueur.get(i).getNom
11                               ());
12            System.out.println("\nQue voulez-vous faire ? \n 1. Echanger des jetons \n
13                               2. Jouer :");

```

```

12
13     while (!scanner.hasNextInt()) {
14         System.out.println("Erreur : veuillez entrer un entier valide !");
15         scanner.next();
16         System.out.println("Entrez un entier : ");
17     }
18
19     int choix1 = scanner.nextInt();
20
21     if (choix1 == 1) {
22         this.listeJoueur.get(i).getChevalet().echangerJetons();
23     } else {
24         this.listeJoueur.get(i).getChevalet().utiliser_jetons();
25         plateau effacerMemoire();
26         this.listeJoueur.get(i).jouer();
27
28         for (int w = 0; w < plateau.TAILLE; w++) {
29             int c = plateau.getMemoireVive(w, 0);
30             int v = plateau.getMemoireVive(w, 1);
31         }
32
33         if (this.verification()) {
34             this.miseAJourScore(this.listeJoueur.get(i));
35             plateau effacerMemoire();
36         }
37
38         if (this.listeJoueur.get(i).getChevalet().getNombreJetons() == 0) {
39             System.out.println("Joueur " + this.listeJoueur.get(i).getNom() + "
40                 limin");
41             this.listeJoueur.remove(i);
42         }
43
44         if (!this.verification()) {
45             System.out.println("\nMauvais coup, jouez autrement\n");
46
47             for (int x = 0; x < plateau.TAILLE; x++) {
48                 int k = plateau.getMemoireVive(x, 0);
49                 int z = plateau.getMemoireVive(x, 1);
50
51                 if (k != -1 && z != -1) {
52                     Jeton jeton = new Jeton(plateau.getCase(k, z));
53                     this.listeJoueur.get(i).getChevalet().listeJetons.add(jeton);
54                     plateau.modifierCase(k, z, 0);
55                 }
56             }
57
58             plateau effacerMemoire();
59         }
60
61         if (sac_jetons.getTaille() == 0) {
62             System.out.println("Pouvez-vous encore jouer? (1=Oui/0=Non) :");
63
64             while (!scanner.hasNextInt()) {
65                 System.out.println("Erreur : veuillez entrer un entier valide !")
66             }

```

```

65         scanner.next();
66         System.out.print("Entrez un entier : ");
67     }
68
69     int choix = scanner.nextInt();
70     if (choix == 0) {
71         couppossible = false;
72     }
73 }
74 }
75 }
76
77     partieEnCours = verifierConditionsContinuerPartie();
78 }
79
80 gererFinDePartie();
81 }

```

La méthode `verification()` permet de valider les placements des jetons. Son but principal est de vérifier si tous les groupes de jetons posés sont valides, en ligne ou en colonne. Pour cela, elle utilise les méthodes `verif_ligne()` et `verif_col()` afin d'analyser localement les séquences.

La méthode parcourt chaque position enregistrée dans la mémoire vive du plateau. Pour chaque position, elle vérifie que la case n'est pas vide. Elle utilise ensuite `verif_ligne()` pour contrôler la validité horizontale et `verif_col()` pour la validité verticale. Les vérifications sont optimisées en réutilisant les résultats précédents lorsque c'est possible. Un tableau interne nommé `presance` est utilisé pour détecter les connexions adjacentes. Si une ligne ou une colonne est incorrecte, un compteur d'erreurs est incrémenté.

À la fin de l'analyse, si aucune erreur n'a été détectée, la méthode retourne `true`, sinon elle retourne `false`.

Code Java de la méthode `verification()` :

```

1 public boolean verification() {
2     int verif = 0;
3     boolean verifLigne;
4     boolean verifCol;
5     int i_prec = -1; // Initialisation
6     int j_prec = -1; // Initialisation
7     boolean verifLigne_prec = false; // Initialisation
8     boolean verifCol_prec = false; // Initialisation
9     boolean connexionHorizontale;
10    boolean connexionVerticale;
11
12    // Parcours des positions en mmoire vive
13    for (int x = 0; x < plateau.TAILLE; x++) {
14        int i = plateau.getMemoireVive(x, 0);
15        int j = plateau.getMemoireVive(x, 1);
16
17

```

```

18
19     if (i != -1 && j != -1) {
20         // Vrifier si on peut rutiliser les rsultats prcdents
21         if (x != 0 && i_prec == i) {
22             verifLigne = verifLigne_prec;
23             verif_ligne(i, j);
24         } else {
25             verifLigne = verif_ligne(i, j);
26         }
27
28         if (x != 0 && j_prec == j) {
29             verifCol = verifCol_prec;
30             verif_col(i, j); // Met jour presance mme si on ne change pas
                               // verifCol
31         } else {
32             verifCol = verif_col(i, j);
33         }
34
35         connexionHorizontale = this.presance[0] || this.presance[1];
36         connexionVerticale = this.presance[2] || this.presance[3];
37
38         boolean ligneValide = verifLigne || connexionVerticale;
39         boolean colonneValide = verifCol || connexionHorizontale;
40
41         if (!ligneValide || !colonneValide) {
42             verif++;
43         }
44         // Sauvegarder les valeurs pour la prochaine itration
45         i_prec = i;
46         j_prec = j;
47         verifLigne_prec = verifLigne;
48         verifCol_prec = verifCol;
49     }
50 }
51
52 if(verif==0) return true;
53 return false;
54 }

```

Description des méthodes auxiliaires :

La méthode `verif_col(int l, int c)` vérifie la validité d'une colonne en partant de la position (l, c). Elle cherche les limites hautes et basses de la séquence continue de jetons, calcule la somme des valeurs présentes sur cette colonne, puis compare cette somme aux numéros attendus du paquet de cartes. Elle met également à jour le tableau `presance` pour indiquer une connexion verticale.

La méthode `verif_ligne(int l, int c)` fonctionne de manière similaire, mais pour les lignes. Elle vérifie la validité d'une ligne en partant de la position (l, c), cherche les limites gauche et droite de la séquence continue de jetons, calcule la somme des valeurs présentes sur cette ligne, puis la compare à la valeur attendue. Elle met à jour le tableau `presance` pour signaler une connexion horizontale.

Ci-dessous, un tableau représentant l'ensemble des cas possibles présents dans le jeu.

Vérif Ligne	Vérif Colonne	Prés. G.	Prés. D.	Prés. H.	Prés. B.	Résultat
true	false	true	false	false	false	true
true	false	false	true	false	false	true
true	false	false	false	true	false	false
true	false	false	false	false	true	false
false	true	true	false	false	false	false
false	true	false	true	false	false	false
false	true	false	false	true	false	true
false	true	false	false	false	true	true
true	true	true	true	false	false	true
true	true	false	false	true	true	true
false	false	false	false	false	false	false
false	false	true	false	false	false	false
false	false	false	true	false	false	false
false	false	false	false	true	false	false
false	false	false	false	false	true	false

5 UML après code

Après avoir réalisé le code, nous nous sommes rendus compte que certaines choses étaient à revoir dans la conception de notre diagramme de classe, certaines méthodes exposées dans le diagramme de classe initialement étaient dès lors redondantes, n'avaient pas d'utilité ou bien alors n'étaient pas présentes.

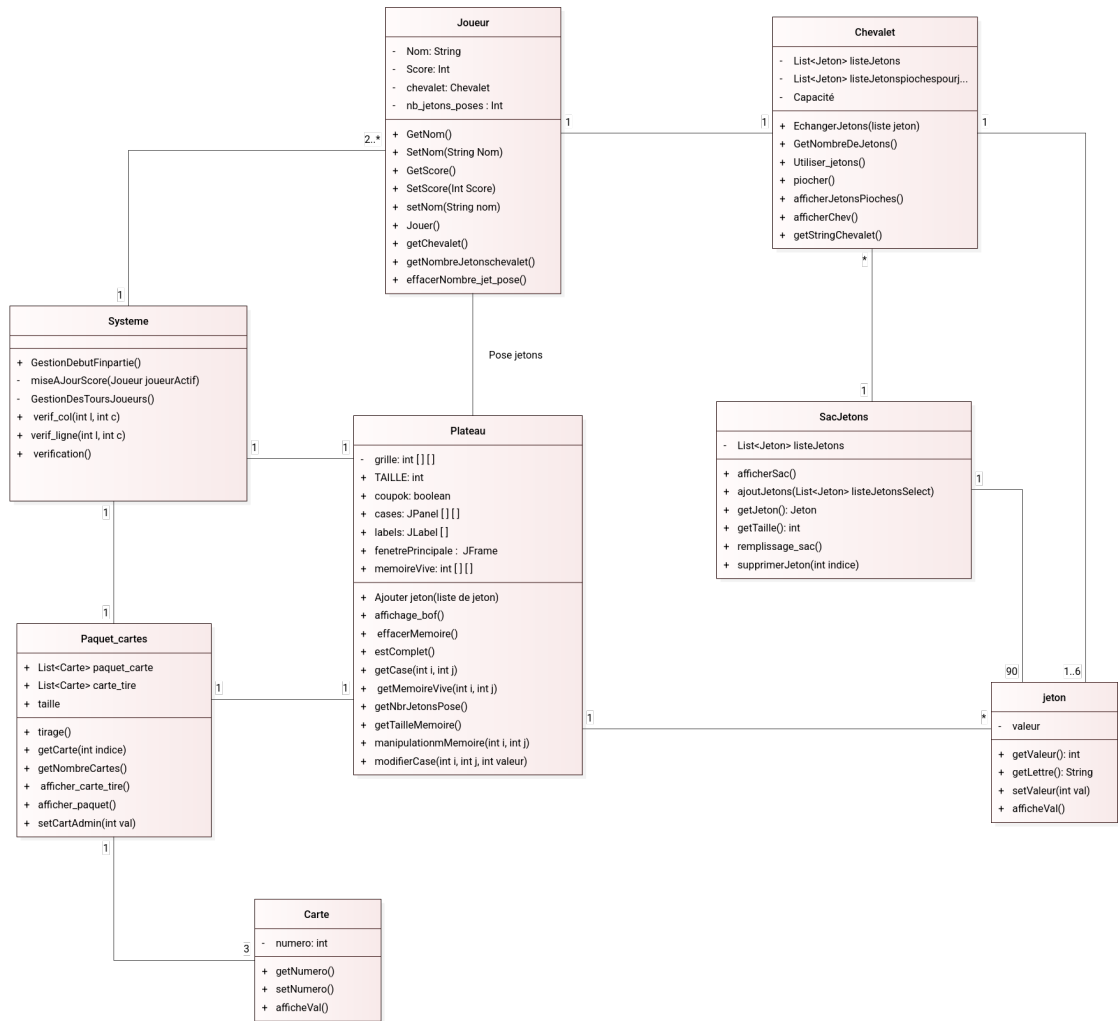


Figure 1.5 – *Diagramme de Classe final*

6 Test

Nous allons désormais passer aux tests afin de déterminer les limites d'un tel projet.

Dans un premier temps, l'utilisateur est amené à choisir le nombre de joueurs qui souhaitent participer au jeu. Après avoir mis le nombre de joueur, c'est le tour du premier joueur. Il peut s'il le souhaite échanger ses jetons ou bien jouer.


```
4 Au tour du joueur Amr
Que voulez-vous faire ?
1. Echanger des jetons
2. Jouer :
1
La pioche :
[15]
[15]
Selectionner vos jetons
Saisir le nombre de jetons que vous voulez echanger :
1 : [17]
2 : [19]
3 : [12]
4 : [13]
5 : [ 9]
6 : [12]
7 : [ 3]
8 : [ 2]
9 : [ 3]
4
Erreur : Depassement capacite(9)
Saisir le nombre de jetons que vous voulez echanger :
```

Figure 1.8

```
4 3 Saisir l'indice du jeton :
1 : [17]
2 : [19]
3 : [12]
4 : [13]
5 : [ 9]
6 : [12]
7 : [ 3]
8 : [ 2]
9 : [ 3]
1
Saisir l'indice du jeton :
1 : [19]
2 : [12]
3 : [13]
4 : [ 9]
5 : [12]
6 : [ 3]
7 : [ 2]
8 : [ 3]
2
Saisir l'indice du jeton :
1 : [19]
```

Figure 1.9

C'est ensuite le tour du joueur suivant. Il peut lui aussi jouer ou bien échanger des jetons. Le joueur suivant choisit ainsi de jouer, il doit choisir un nombre de jetons à utiliser. S'il

souhaite utiliser un jeton déjà choisi, une erreur survient et il lui est demandé de choisir un jeton non déjà pioché.

```
Au tour du joueur   Mouad
Que voulez-vous faire ?
1. Echanger des jetons
2. Jouer :
2
Quels jetons voulez-vous utiliser, donner l'indice entre 1 et 9?, 0 pour sortir
:
1
1
Erreur choisissez un jeton qui n'est pas déjà sélectionné
2
3
0
Sortie du choix des jetons.
[ 9]
[ 6]
[14]
Quelle pièce voulez vous choisir ? (de 1 à 3, 0 pour sortir) : 1
Ecrivez la ligne et colonne : 1
1
true
```

Figure 1.10

Il est ensuite affiché les jetons que le joueur a décidé de choisir et désormais il doit choisir lesquels poser et la position à laquelle il souhaite le faire. Lorsque le joueur a déposé toutes les pièces qu'il souhaite, une vérification a lieu et il lui est demandé s'il souhaite arrêter la partie. S'il a bien ou mal joué, c'est au tour du joueur suivant. Les jetons non utilisés sont dès lors ramenés dans le chevalet du joueur respectif.

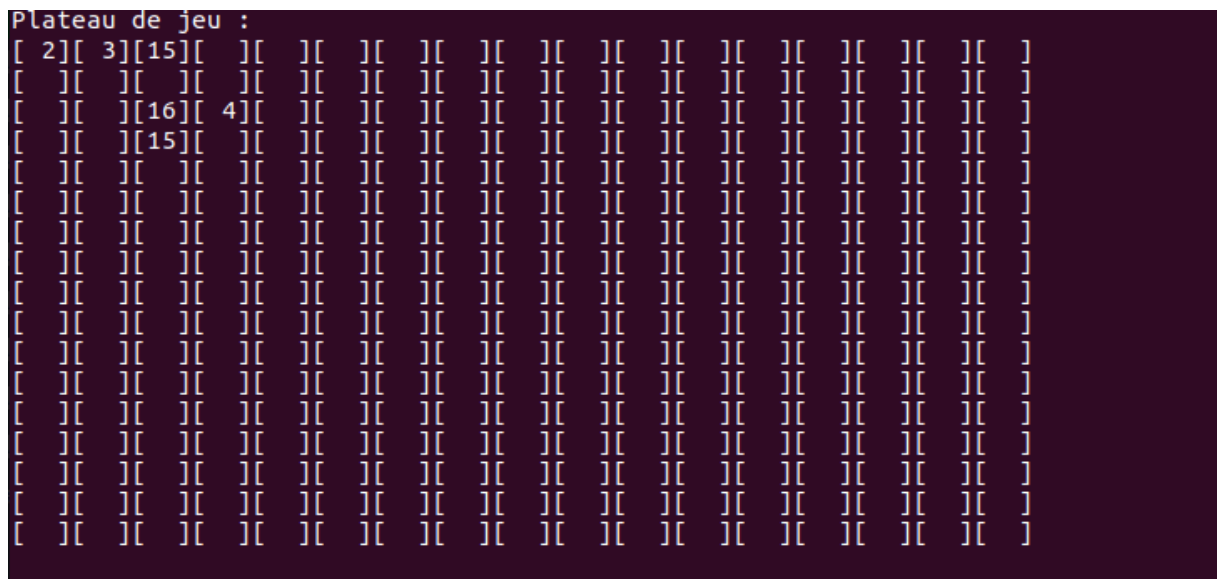


Figure 1.13

Ainsi la partie continue jusqu'à ce qu'un joueur souhaite arrêter ou bien encore si le plateau est plein. Par ailleurs si un joueur n'a plus de jetons il est alors éliminé de la partie.

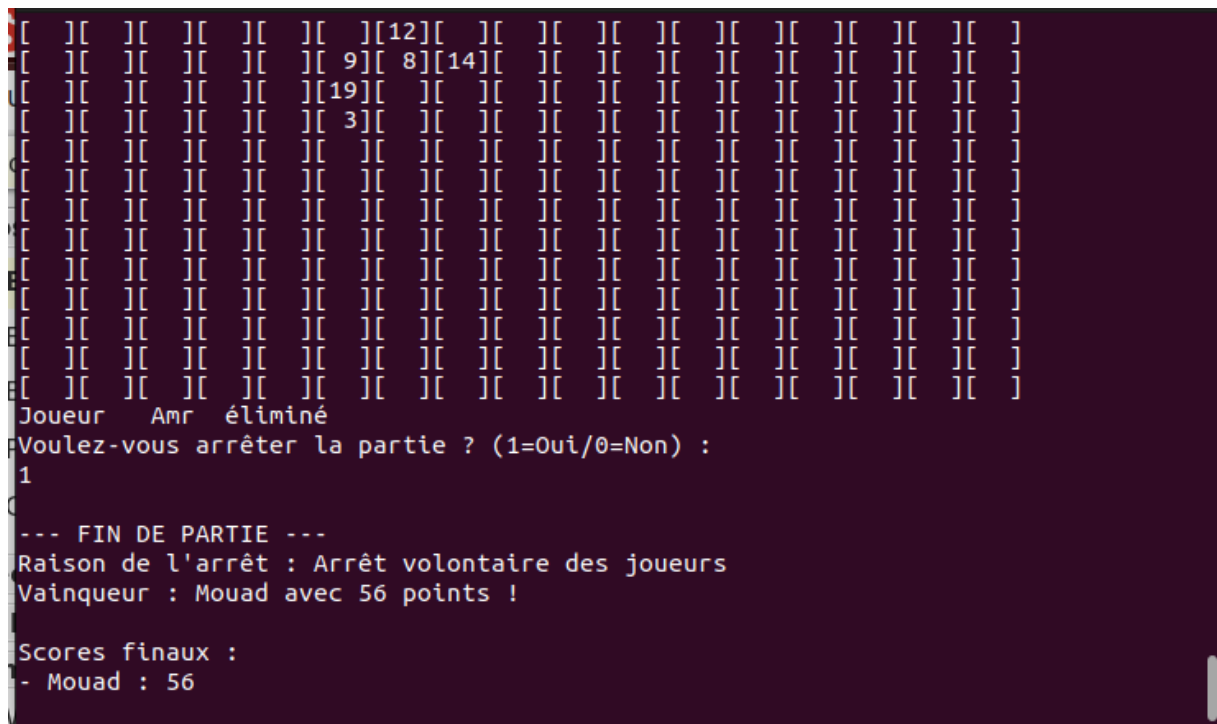


Figure 1.14

7 Conclusion

Pour conclure, nous avons apprécié de pouvoir travailler sur ce projet autour du jeu des chiffres croisés. Bien que nous ne partions pas de zéro en Java, nous n'avions encore jamais mené un projet aussi complet dans ce langage. Ce fut donc l'occasion d'approfondir nos connaissances, de consolider nos bases et de mettre en pratique les concepts de la programmation orientée objet dans un cadre concret et stimulant. Ce projet nous a permis de mieux comprendre la structuration d'un programme Java, la gestion des classes, des interactions entre objets, et d'aborder la conception d'une interface graphique.

Concernant le travail en groupe, nous pensons avoir su gérer efficacement le temps imparti en travaillant de manière régulière et organisée. Nous étions très motivés à l'idée de voir fonctionner ce que nous avions d'abord imaginé dans notre diagramme UML. Même si nous avons parfois dû travailler à distance, la mise en place d'un espace de partage collaboratif nous a permis de répartir les tâches naturellement, tout en nous soutenant mutuellement à chaque étape du projet.

Bibliography

- [1] *Class Scanner*. Documentation officielle de Java. <https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html>
- [2] *Interface List*. Documentation officielle de Java. <https://docs.oracle.com/javase/8/docs/api/java/util/List.html>
- [3] *Class ArrayList*. Documentation officielle de Java SE 8. <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- [4] *Swing Tutorial – Creating a GUI With Swing*. <https://docs.oracle.com/javase/tutorial/uiswing/>
- [5] *Java AWT Package Summary*. <https://docs.oracle.com/javase/8/docs/api/java/awt/package-summary.html>
- [6] Learn Java Swing Programming. Playlist YouTube. Consulté en avril 2025. https://www.youtube.com/playlist?list=PL3bGLnkkGnuV699lP_f9DvxyK5lMFpq6U